

Laboratorio 2

Diego Stefanini — prof. Patrizio Dazzi, Luca Ferrucci, a.a. 2026-27

Indice

1 L'ambiente di lavoro: UNIX e primi programmi C	3
1.1 Il terminale e il filesystem	3
1.2 Dal sorgente al programma	4
1.3 Il primo programma C	5
1.4 Lavorare con i file	7
1.5 Flussi: redirectione e pipe	8

1 L'ambiente di lavoro: UNIX e primi programmi C

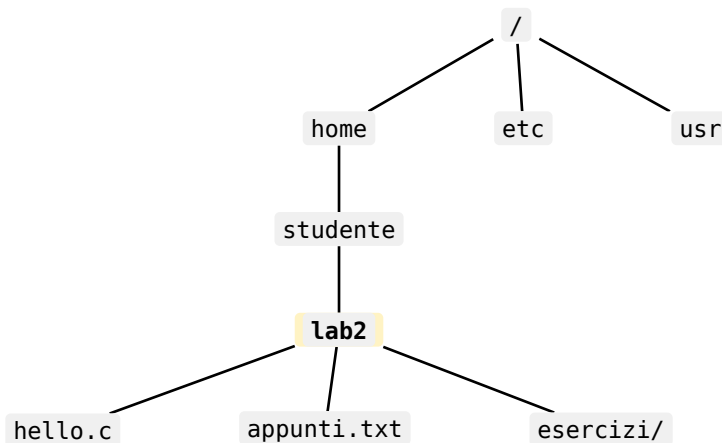
1.1 Il terminale e il filesystem

1.1.1 Terminale e shell

```
$ pwd
/home/studente/lab2
$ ls
hello.c  appunti.txt
```

- **Terminale:** interfaccia testuale per dialogare con il sistema.
- **Shell** (bash, zsh): il programma dentro il terminale che interpreta i comandi.
- Il **prompt** (`$`) dice che la shell aspetta un comando. Non fa parte del comando e non è sempre `$`.

1.1.2 Il filesystem



Il filesystem è un **albero**. Parte da `/`, la **root**: ogni file o directory si raggiunge con un **percorso**. `/` separa un livello dal successivo.

File e directory sono oggetti diversi ma stanno nello stesso spazio dei nomi.

In giallo la **directory corrente**.

- `.` = la directory corrente
- `..` = la directory padre

Comando	Descrizione
<code>pwd</code>	<i>print working directory</i> : stampa in quale cartella sei
<code>ls</code>	elenca il contenuto della directory corrente
<code>ls -l</code>	elenco in formato lungo (permessi, proprietario, dimensione, data)
<code>cd</code>	<i>change directory</i> : navigo in una directory
<code>mkdir</code>	<i>make directory</i> : crea una directory
<code>echo \$?</code>	restituisce il valore di ritorno dell'ultimo comando eseguito

1.1.2.1 Percorsi assoluti e relativi

Relativo: parte dalla directory corrente (es. `esercizi/es1.c`). **Assoluto:** parte dalla root `/` (es. `/home/studente/lab2`).

```
$ cd esercizi
$ pwd
/home/studente/lab2/esercizi
```

```
$ cd /home/studente
$ pwd
/home/studente
```

```
$ pwd
/home/studente/lab2
$ cd ..      # vado nel padre
$ pwd
/home/studente
```

1.1.2.2 La home e i file nascosti

`~` = la **home** dell'utente corrente. `cd` senza argomenti porta di solito lì.

Un nome che inizia con `.` è **nascosto**: `ls` non lo mostra, `ls -a` sì.

```
$ cd ~
$ pwd
/home/studente
```

```
$ ls
appunti.txt hello.c
$ ls -a
. .. .bashrc appunti.txt hello.c
```

«Nascosto» è solo una **convenzione sul nome**, non un tipo speciale di file.

1.1.2.3 Leggere `ls -l`

<code>d</code>	<code>rwXr-xr-x</code>	<code>studente</code>	<code>studenti</code>	<code>64</code>	<code>Sep 15 09:10</code>	<code>esercizi</code>
tipo: d = directory, - = file	permessi	proprietario	gruppo	dimensione (byte)	ultima modifica	nome

1.1.3 Com'è fatto un comando

<code>gcc</code>	<code>-Wall</code>	<code>hello.c</code>	<code>-o hello</code>
comando	opzione	argomento	opzione + argomento

Le **opzioni** modificano il comportamento del comando. Se non ricordi come si usa: `gcc --help` (sintesi rapida) oppure `man gcc` (manuale UNIX).

1.1.3.1 Chi riceve gli argomenti?

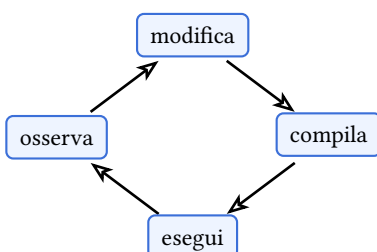
La shell **spezza la riga in parole usando gli spazi** e avvia il programma passandogli gli argomenti. Se un nome contiene spazi, le **virgolette** lo tengono insieme come un solo argomento:

```
$ touch "appunti lezione.txt"    # un file, non due
$ cat "appunti lezione.txt"
```

Vedremo come un programma C legge questi argomenti con `argc` e `argv`, i parametri di `main`.

1.2 Dal sorgente al programma

1.2.1 Il ciclo di lavoro

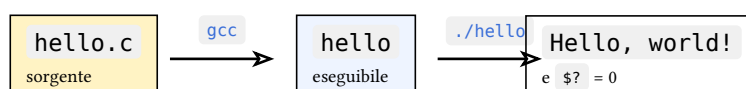


Il programma non nasce nell'editor: nasce da questo ciclo, ripetuto.

modifica ≠ **compilazione** ≠ **esecuzione**. Se cambi il sorgente e non ricompili, stai eseguendo il vecchio programma.

1.2.2 Compilare ed eseguire

C è un linguaggio **compilato**: il sorgente `.c` va trasformato in un file binario prima di poterlo eseguire.



```
$ gcc hello.c # se va tutto bene non scrive niente
$ ls
a.out hello.c # senza -o l'eseguibile si chiama a.out
$ gcc -Wall -Wextra -std=c11 hello.c -o hello # warning attivi, standard C11, nome "hello"
$ ./hello
Hello, world!
$ echo $?
0
```

Pezzo	Significato
-o hello	dà il nome all'eseguibile
-Wall -Wextra	attiva i warning: non sono rumore , spesso segnalano un errore vero
-std=c11	standard del C (esistono C89, C95, C99, C11, C17, C23). gcc = GNU C Compiler
./hello	«esegui hello che sta nella directory corrente ». La shell cerca i comandi in un elenco di directory, e quella corrente non c'è per forza
echo \$?	stampa il valore restituito dall'ultimo programma. 0 = terminato correttamente

1.2.3 Quando qualcosa va storto

```
int main(void) {
    puts("Hello, world!") // manca ;
    return 0;
}
```

```
hello.c:4:26: error: expected ';' before 'return'
```

Errore di compilazione	Errore di esecuzione
il programma non viene costruito	l'eseguibile esiste ma si comporta male o termina in modo inatteso

File, riga, messaggio: usali. Correggi **dall'alto verso il basso**: un solo errore può generare molti messaggi dopo.

3 errori banali che fanno perdere tempo

1. essere nella directory sbagliata;
2. modificare un file e compilarne un altro;
3. dimenticare di ricompilare.

Diagnosi minima

1. sono nella directory giusta?
2. il sorgente esiste?
3. ci sono errori o warning?
4. l'eseguibile è stato creato?
5. sto eseguendo quello giusto?

Routine utile: `pwd` → `ls` → `gcc ...` → `./programma`.

1.3 Il primo programma C

1.3.1 I pezzi

```
#include <stdio.h>

int main(void) {
    puts("Hello!");
}
```

1. `#include <stdio.h>`: rende disponibili le funzioni di input/output (come `puts`)

```
    return 0;
}
```

2. `main` : da qui parte il programma. Prende niente (`void`), restituisce un intero (`int`)
3. `puts("Hello!")` : un'operazione
4. `return 0` : termina `main` e restituisce 0 al sistema operativo

Le **graffe** `{ }` delimitano un blocco. I commenti (`//` fino a fine riga, `/*` su più righe `*/`) sono per le persone: il compilatore li ignora. Indentazione e nomi sensati non sono obbligatori in C, ma il programma deve essere leggibile.

1.3.2 Stampare: `puts` e `printf`

Funzione	Cosa fa
<code>puts("Laboratorio 2");</code>	stampa una stringa e va a capo
<code>printf("Risultato: %d\n", 42);</code>	stringa formattata : <code>%d</code> viene sostituito dal valore. Non va a capo da sola : serve <code>\n</code>

`printf` è una funzione a **numero variabile di parametri**: il primo è la stringa di formato, gli altri i valori da inserire al posto di `%d`, `%s`, ... Il placeholder deve essere compatibile con il tipo del valore.

1.3.2.1 Dentro un placeholder

% 0 8 .2 f
 inizio flag: zeri iniziali larghezza minima precisione: 2 decimali tipo: double

`printf("%08.2f", 3.14);` → `00003.14` — forma generale: `%[flags][width][.precision][length]specifier`

Interi		Altri tipi	
<code>%d</code> , <code>%i</code>	int con segno	<code>%f</code>	double
<code>%u</code>	unsigned int	<code>%e</code> , <code>%E</code>	notazione scientifica
<code>%x</code> , <code>%X</code>	esadecimale	<code>%g</code> , <code>%G</code>	formato compatto
<code>%o</code>	ottale	<code>%c</code>	carattere
<code>%ld</code>	long	<code>%s</code>	stringa
<code>%lld</code>	long long	<code>%p</code>	puntatore
<code>%zu</code>	size_t	<code>%%</code>	il carattere %

1.3.3 Mettiamo insieme i pezzi

```
#include <stdio.h>

int main(void) {
    int x = 21; // una
    variabile

    puts("Laboratorio 2");
    puts("Lezione 1");
    printf("2 * %d = %d\n", x, 2 * x); // due
```

```
$ gcc -Wall -Wextra -std=c11
  lezione01.c -o lezione01
$ ./lezione01
Laboratorio 2
Lezione 1
2 * 21 = 42
```

```
placeholder
```

```
    return 0;  
}
```

1.4 Lavorare con i file

1.4.1 Creare: `mkdir` e `touch`

```
$ mkdir esercizi      # argomento = nome  
$ touch appunti.txt   # file vuoto
```

`touch` in realtà nasce per **cambiare i timestamp** di un file. Se il file non esiste lo crea: la creazione è una conseguenza utile, non lo scopo.

1.4.2 I timestamp

Per ogni file UNIX tiene tre marcature temporali. `stat file` le mostra, insieme agli altri metadati.

Metadato	Significato
<code>atime</code>	<i>access time</i> : ultimo accesso al contenuto
<code>mtime</code>	<i>modification time</i> : ultima modifica del contenuto
<code>ctime</code>	<i>change time</i> : ultima modifica dei metadati

`ctime` vuol dire **change** time, **non** «creation time». Alcuni filesystem moderni salvano anche il tempo di creazione (*birth time*), ma non fa parte dei tre timestamp UNIX classici.

A cosa servono: backup incrementali, sincronizzazione, diagnostica, cercare i file modificati di recente, e ricompilare solo ciò che è cambiato. `make` (lo vedremo più avanti) confronta i timestamp per decidere se ricompilare.

Chi aggiorna cosa (✓ = portato all'ora corrente):

Operazione	<code>atime</code>	<code>mtime</code>	<code>ctime</code>
<code>touch file</code>	✓	✓	✓ (cambiare i tempi è già una modifica dei metadati)
<code>touch -a file</code>	✓	—	✓
<code>touch -m file</code>	—	✓	✓
<code>cp</code> : destinazione	✓ solo se creato	✓	✓
<code>cp -p</code> : destinazione	copiato dal sorgente	copiato dal sorgente	✓
<code>cp</code> : sorgente	✓ può, per la lettura	—	—

1.4.3 Copiare, spostare, cancellare

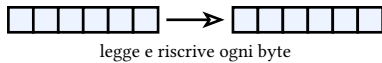
`cp` sorgente destinazione (*copy*): l'originale resta al suo posto.

```
$ cp hello.c copia.c  
$ cp /tmp/ciao.txt ~/hello.txt # copia  
e rinomina
```

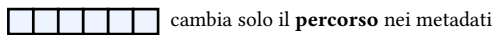
`mv` (*move*): un comando, **due** usi.

```
$ mv copia.c esempio.c    # rinomina  
$ mv esempio.c esercizi/  # sposta
```

copia — $O(N)$



sposta (stesso filesystem) — $O(1)$



Operazione	Costo
copia	$O(N)$
sposta, stesso filesystem	$O(1)$
sposta, altro filesystem	$O(N)$: copia + cancella

Comando	Descrizione
<code>rm</code>	cancella un file (<i>remove</i>)
<code>rmdir</code>	cancella una directory vuota
<code>rm -r</code>	cancella una directory con tutto il contenuto. <code>-r</code> = ricorsivamente : attraversa tutto ciò che c'è dentro

`rm` **non** sposta nel cestino: da terminale la cancellazione è in genere **definitiva**.

1.4.4 Guardare dentro i file

Non serve sempre un editor:

Comando	Descrizione
<code>cat</code>	scrive tutto il contenuto del file su stdout
<code>less</code>	per file lunghi: si scorre avanti e indietro e si cerca, senza riversare tutto sul terminale. q per uscire
<code>head</code> / <code>tail</code>	prime / ultime 10 righe. <code>head -n 5 dati.txt</code> = prime 5

1.4.5 Wildcard: le espande la shell

- `*` = **zero o più** caratteri qualsiasi
- `?` = **esattamente un** carattere

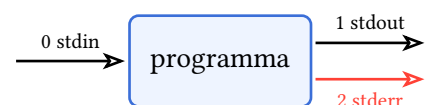
```
$ ls *.c
hello.c main.c prova.c
$ ls prova?.c
prova1.c prova2.c provaA.c
```



1.5 Flussi: redirectione e pipe

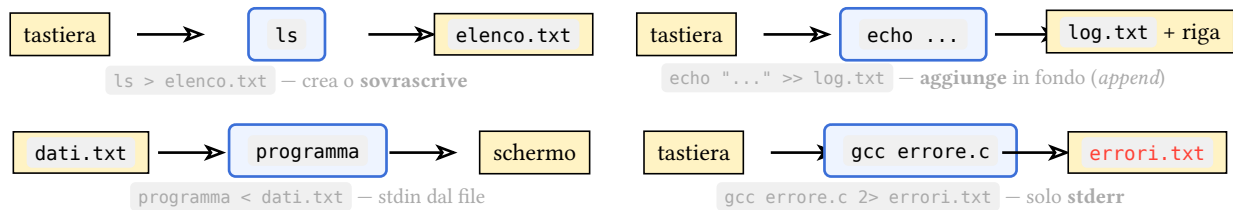
1.5.1 I tre flussi standard

n.	Flusso	Di solito
0	<code>stdin</code> — standard input	dalla tastiera
1	<code>stdout</code> — standard output	sul terminale
2	<code>stderr</code> — standard error	sul terminale



La shell può collegare ciascun flusso a qualcosa di diverso dal terminale.

1.5.2 Redirezioni



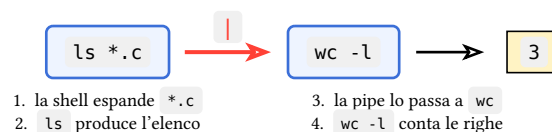
```
$ ls > elenco.txt           # sul terminale non compare niente
$ cat elenco.txt
hello.c
$ echo "prima riga" > log.txt
$ echo "seconda riga" >> log.txt
$ cat log.txt
prima riga
seconda riga
```

Con `<` il programma continua a leggere da *stdin*: è **la shell** che ha collegato *stdin* al file, il programma non se ne accorge.

`2>` redirige il file descriptor 2. Si separa *stderr* da *stdout* perché output normale ed errori hanno significati diversi e spesso si trattano in modo diverso.

1.5.3 La pipe |

Collega lo **stdout** del primo comando allo **stdin** del secondo.



Con un file temporaneo

```
$ ls > tmp.txt
$ wc -l < tmp.txt
```

Con la pipe: niente file in mezzo

```
$ ls | wc -l
$ ls | less      # scorro l'elenco
```

Idea UNIX: piccoli programmi che si combinano per costruire operazioni più complesse.

1.5.4 Micro-esercizio

Partendo dalla home: creare `lezione02`, entrarci, creare `a.c`, `b.c`, `note.txt` vuoti, salvare l'elenco dei soli `.c` in `sorgenti.txt`, aggiungerlo a `note.txt`, contare le righe di `note.txt`.

```
$ mkdir lezione02
$ cd lezione02
$ touch a.c b.c note.txt
$ ls *.c > sorgenti.txt
$ cat sorgenti.txt >> note.txt
$ wc -l < note.txt
2
```